

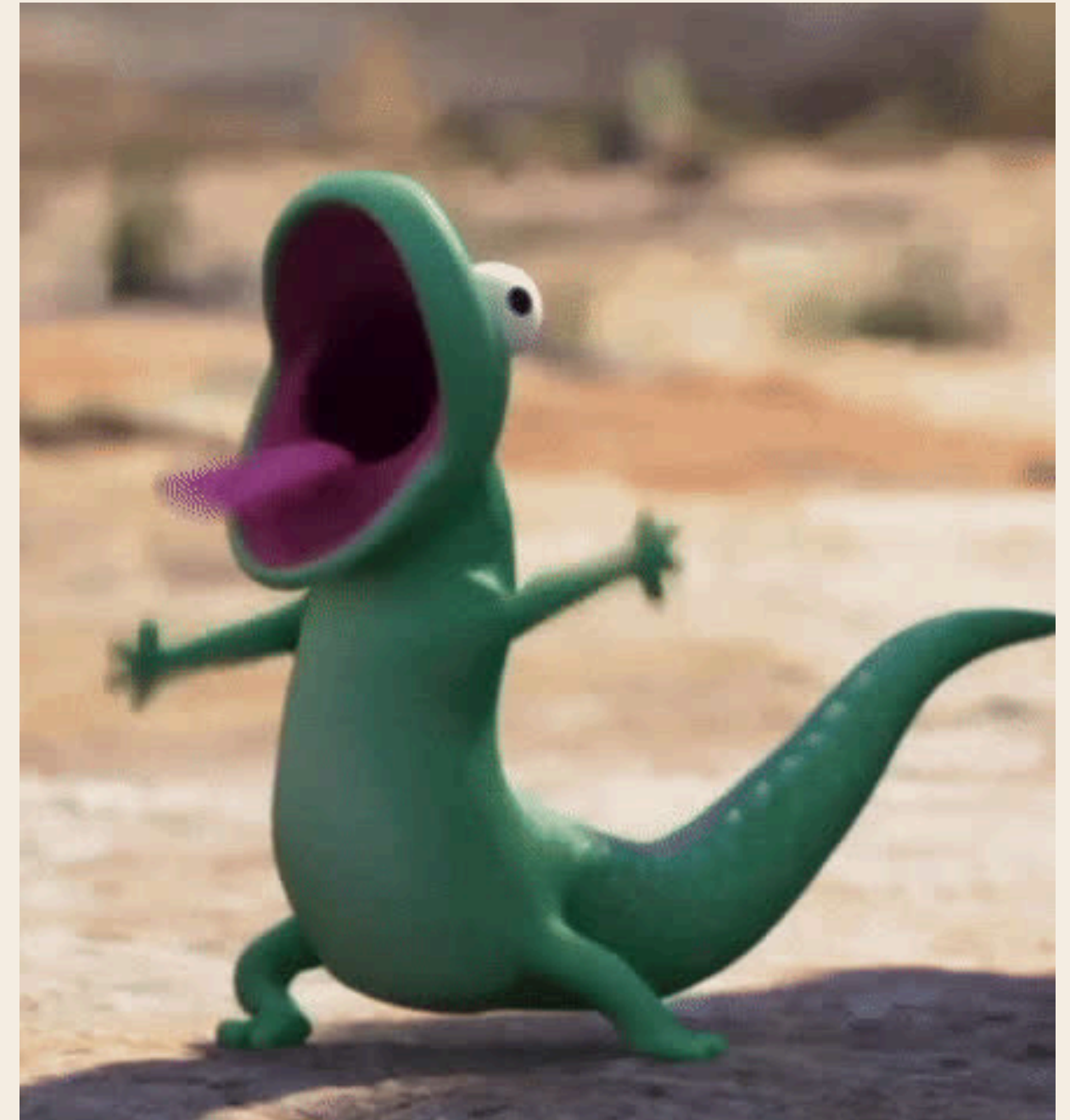
IT1244

Artificial Intelligence: Technology & Impact

Week 10 Tutorial 8

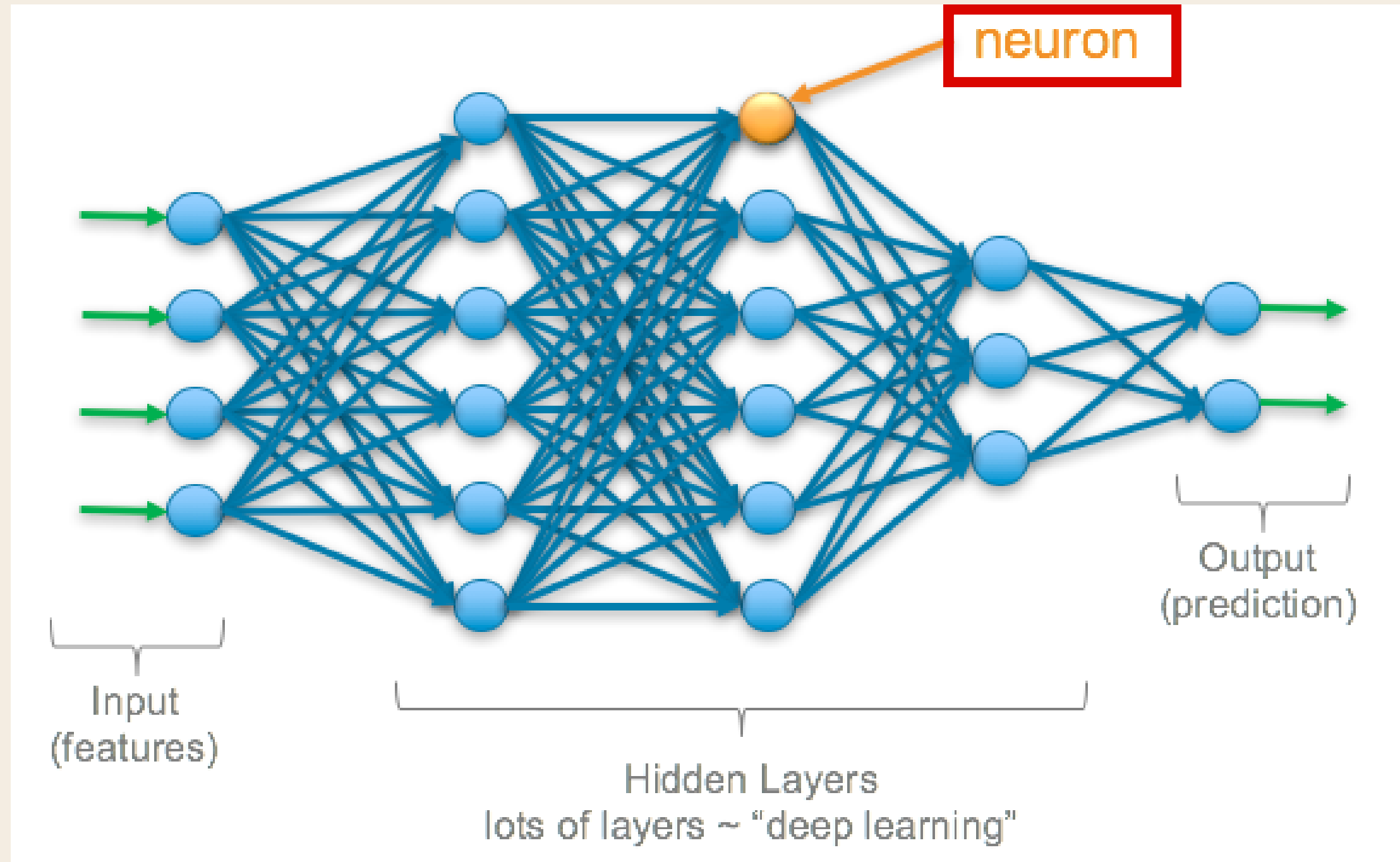
TODAY'S AGENDA

1. Recap of concepts
2. Discussion of tutorial questions
3. Pollev Questions



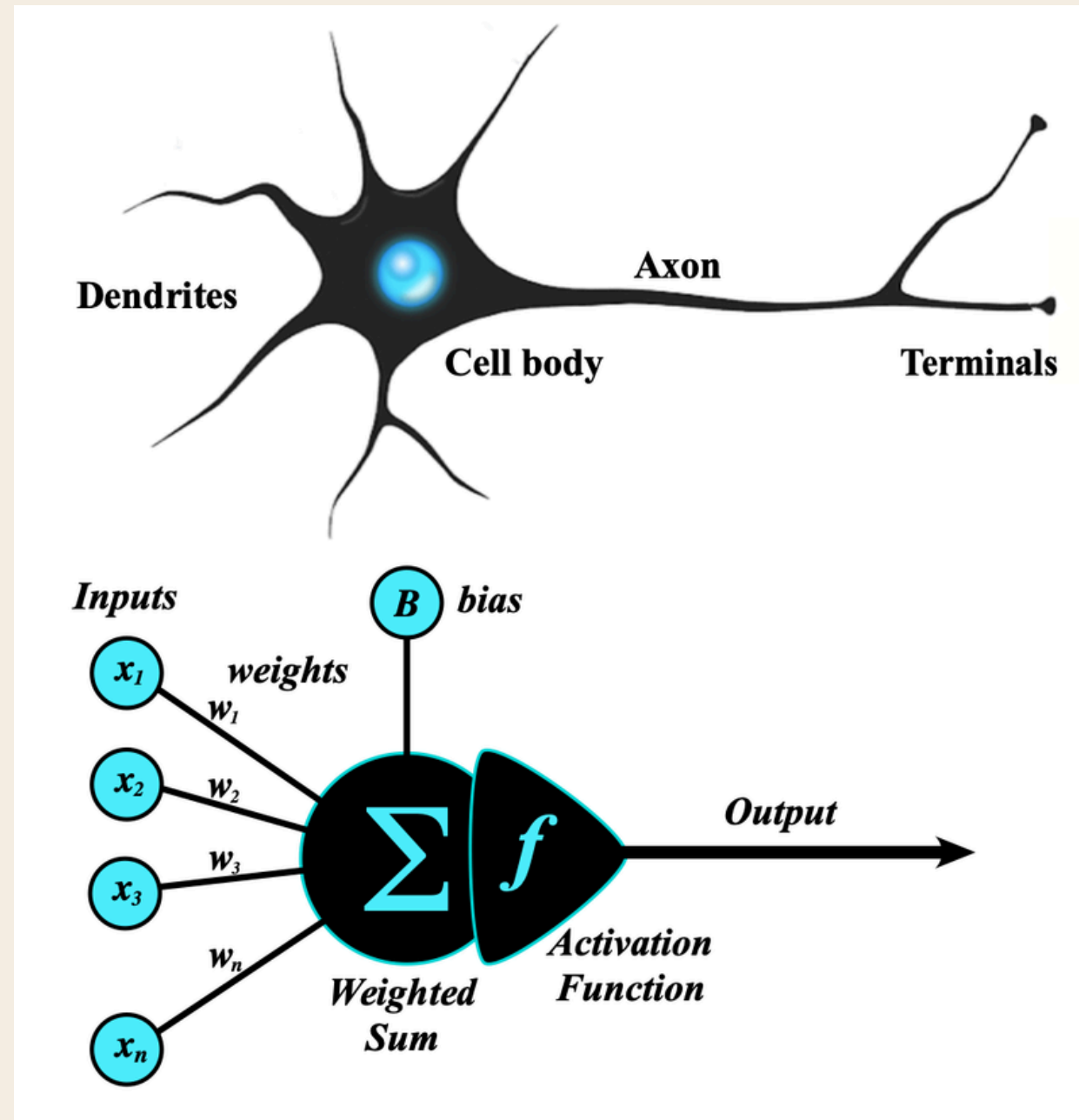
RECAP

ARTIFICIAL NEURAL NETWORK (ANN)



Consists of three main layers: Input, Hidden and Output Layer.

NEURONS IN NEURAL NETWORK



An (Artificial) Neuron consists of:

- Set of Inputs
- One output
- Weights
- Bias
- Activation function

Artificial Neurons were inspired by Biological Neurons

NEURONS IN NEURAL NETWORK

Every neuron performs the following operation:

1. Takes in a set of inputs (≥ 1 input) and bias.

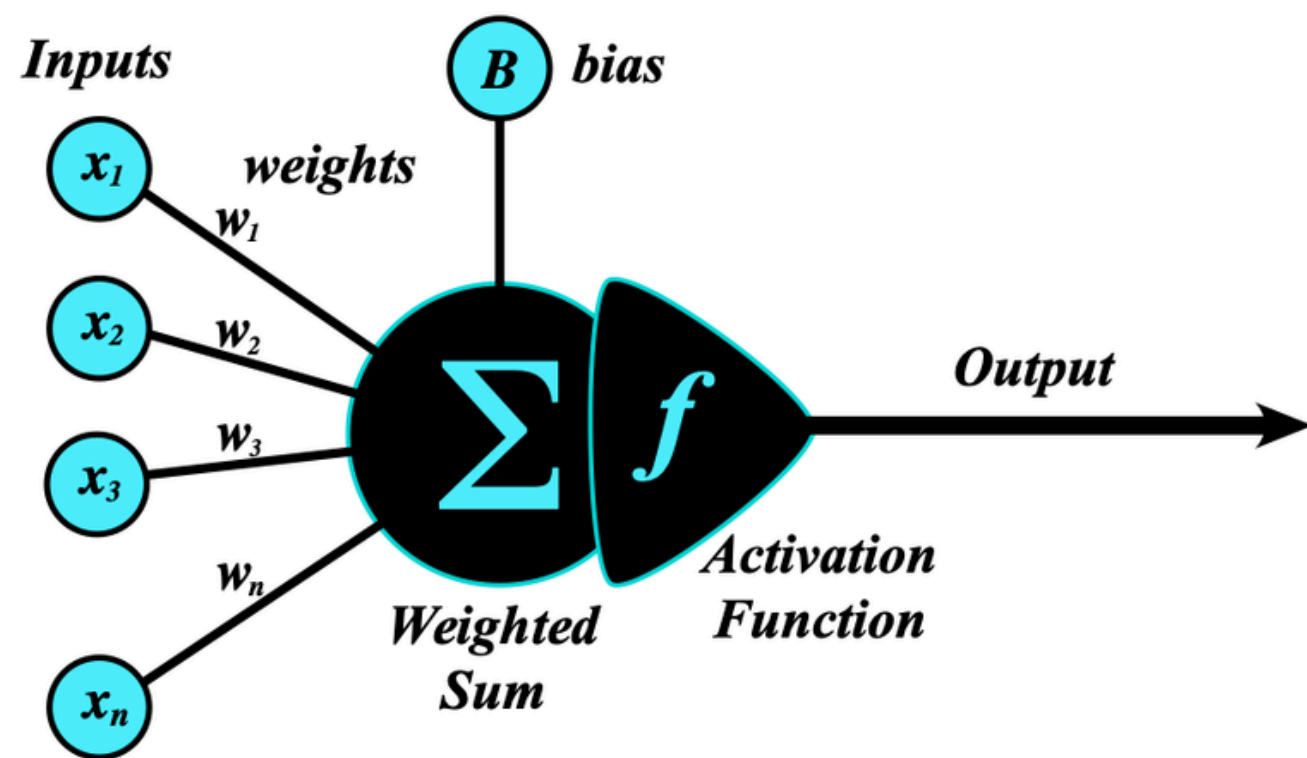
2. Performs linear combination of weights and inputs.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

3. Passes in the weighted sum of inputs into an Activation Function.

$$\hat{y} = f(z)$$

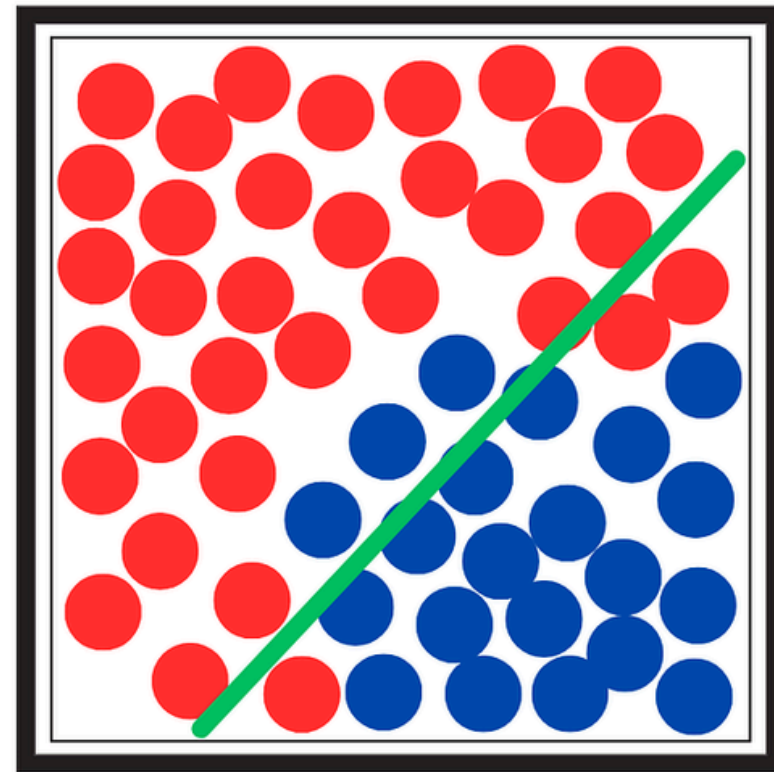
4. Returns a scalar output.



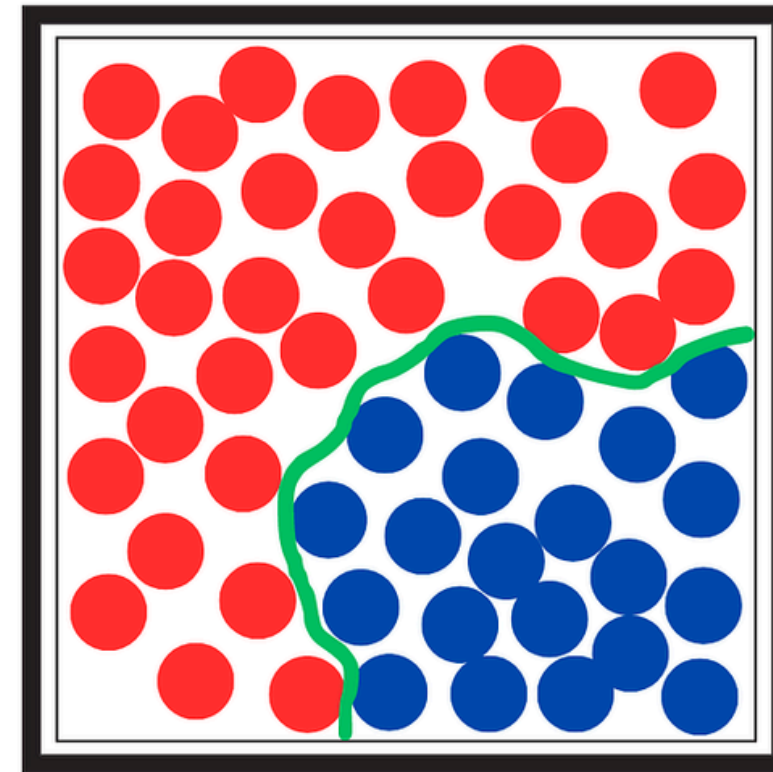
So essentially: $\hat{y} = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$

ACTIVATION FUNCTION

Activation functions are usually used to introduce **non-linearity** to the network



Neural Network without
an Activation Function

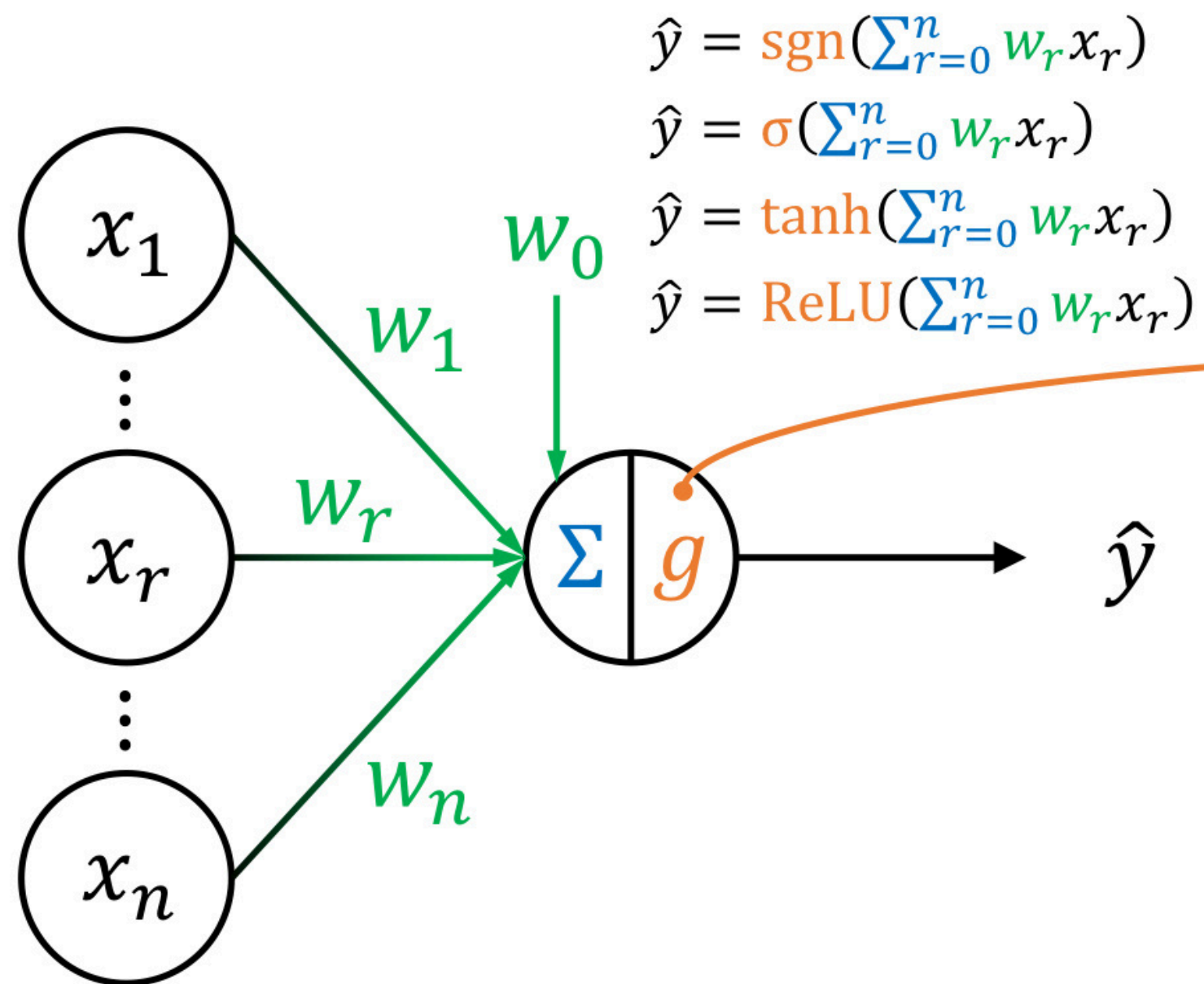


Neural Network with
an Activation Function

However, you can still choose $f(x) = x$ (a linear mapping) as your activation function, though not commonly seen.

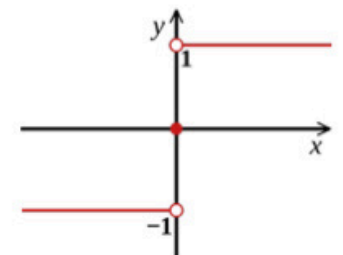
COMMON ACTIVATION FUNCTIONS

Activation Functions



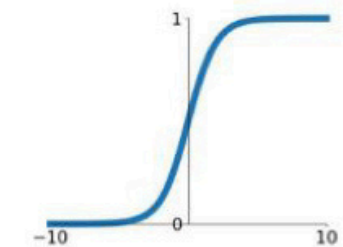
Step

$$\text{sgn}(x) = \begin{cases} +1 & z > 0 \\ -1 & z \leq 0 \end{cases}$$



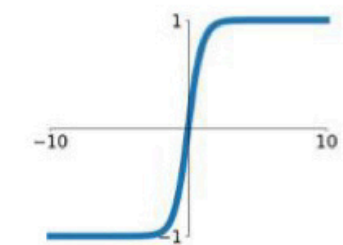
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$

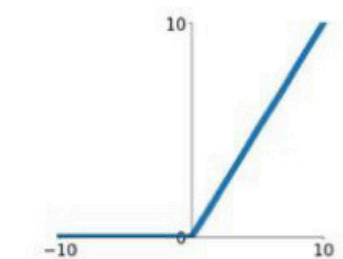


Image Credit:

https://miro.medium.com/max/1400/0*sIJ-gbjlz0zrz8lb.png

MORE ACTIVATION FUNCTIONS

Sigmoid



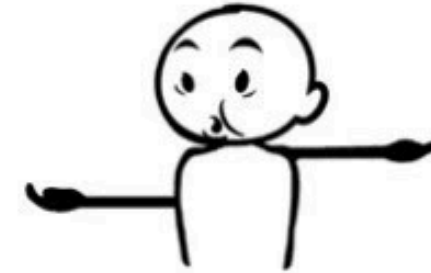
$$y = \frac{1}{1 + e^{-x}}$$

Tanh



$$y = \tanh(x)$$

Step Function



$$y = \begin{cases} 0, & x < n \\ 1, & x \geq n \end{cases}$$

Softplus



$$y = \ln(1 + e^x)$$

ReLU



$$y = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

Softsign



$$y = \frac{x}{(1 + |x|)}$$

ELU



$$y = \begin{cases} \alpha(e^x - 1), & x < 0 \\ x, & x \geq 0 \end{cases}$$

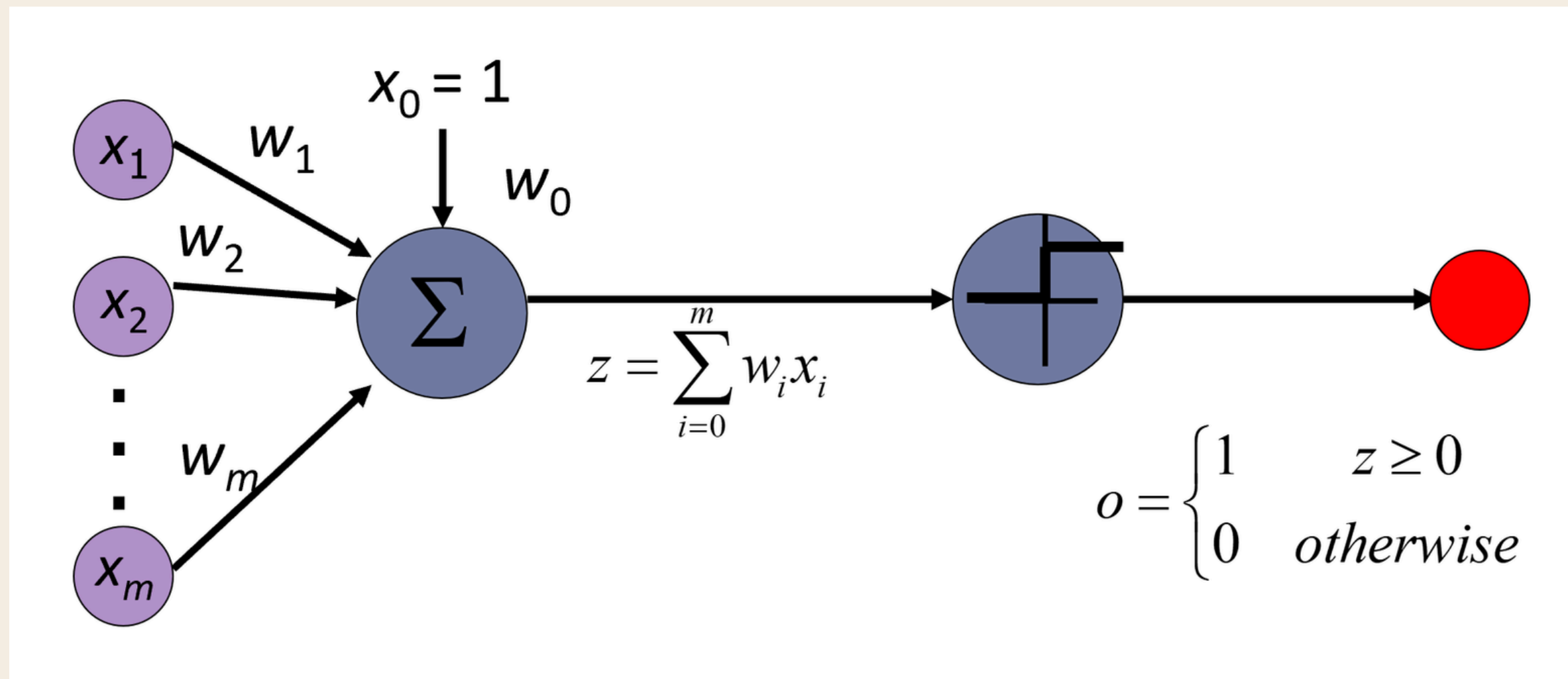
Log of Sigmoid



$$y = \ln\left(\frac{1}{1 + e^{-x}}\right)$$

PERCEPTRON

A perceptron is a special case of a neuron where the activation function is specifically the binary step function (or called Heaviside step function).



Perceptron is historically the first neuron introduced by Frank Rosenblatt in 1958

LAYERS OF NEURAL NETWORK

1. Input Layer

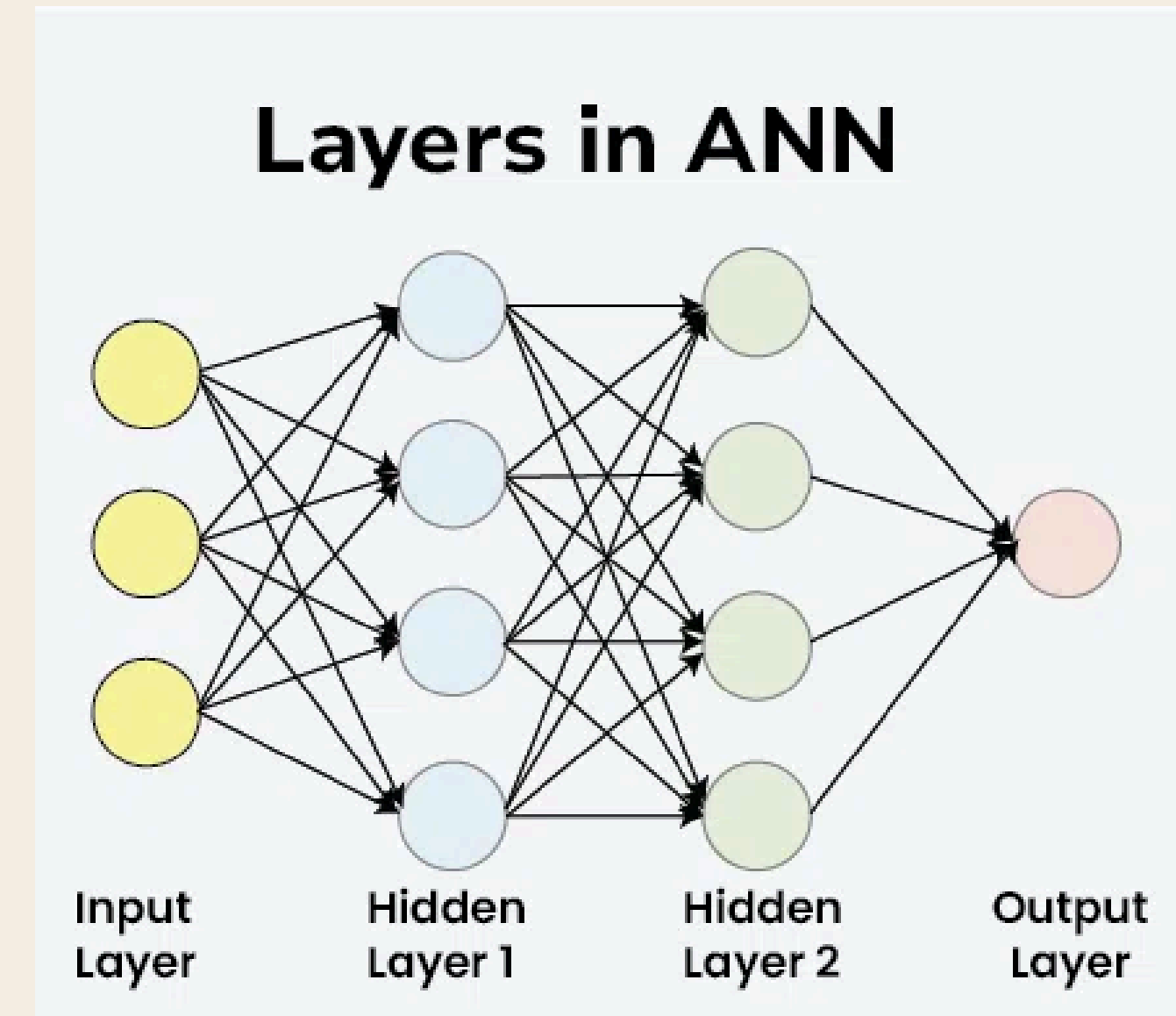
- Does not have neurons → no computation / activation fn.

2. Hidden Layer(s)

- Contains some neurons that all share similar activation fn.
- Can have as many hidden layers as you want

3. Output Layer

- Produces the network's prediction
 - a. **Regression:** Single neuron that outputs continuous value
 - b. **Classification:** # neurons = # total classes (output prob. of each class)



NEURAL NETWORK IN CODE

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),

    nn.Linear(8, 6),
    nn.ReLU(),

    nn.Linear(6, 1),
    nn.Sigmoid()
)
```

We have 4 inputs (excluding bias)

{ Layer 1: 4 Inputs → 8 Outputs
Activation Function: ReLU

{ Layer 2: 8 Inputs → 6 Outputs
Activation Function: ReLU

{ Layer 3: 6 Inputs → 1 Output
Activation Function: Sigmoid

MULTILAYER NEURAL NETWORK

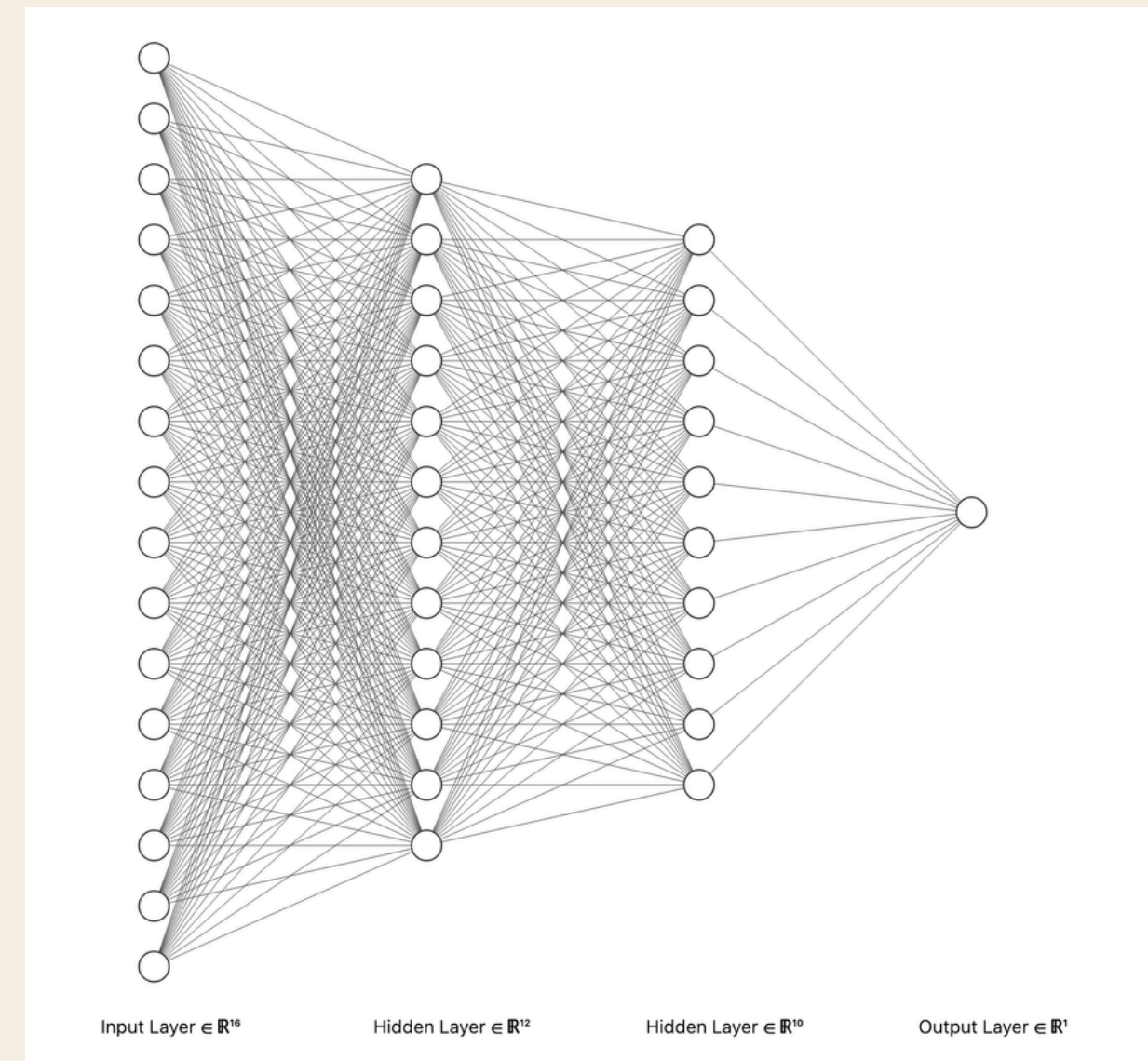
A general term that refers to any Neural Network with **at least one hidden layer**. Can have various architectures (e.g., MLP, CNN, Transformer)

How to generate output? - Feedforward algorithm

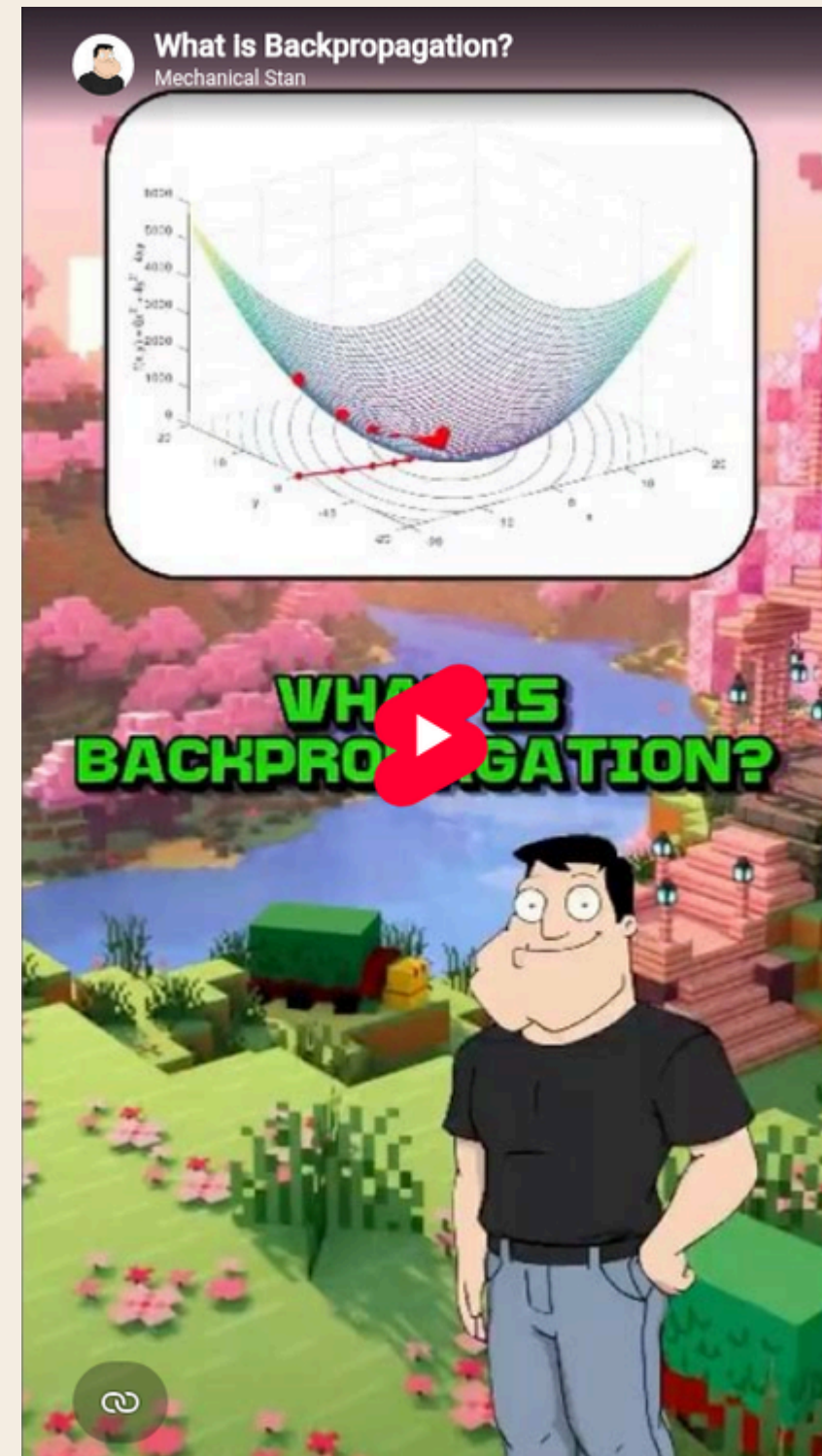
Most of the Multilayer NNs follow Feedforward algorithm, where inputs flow towards the output layer (**Exception: RNN**)

How to train network (find best weights) - Backpropagation

Network is trained with the Backpropagation algorithm, where weight updates are performed starting from the last layer



BACKPROPAGATION (EXTRA-ISH BUT VERY USEFUL!)



<http://youtube.com/shorts/7BXyqq7YwQE>

TRAINING A NEURAL NETWORK

Feedforward → compute prediction → compute loss → **Backpropagation** → update weights → repeat

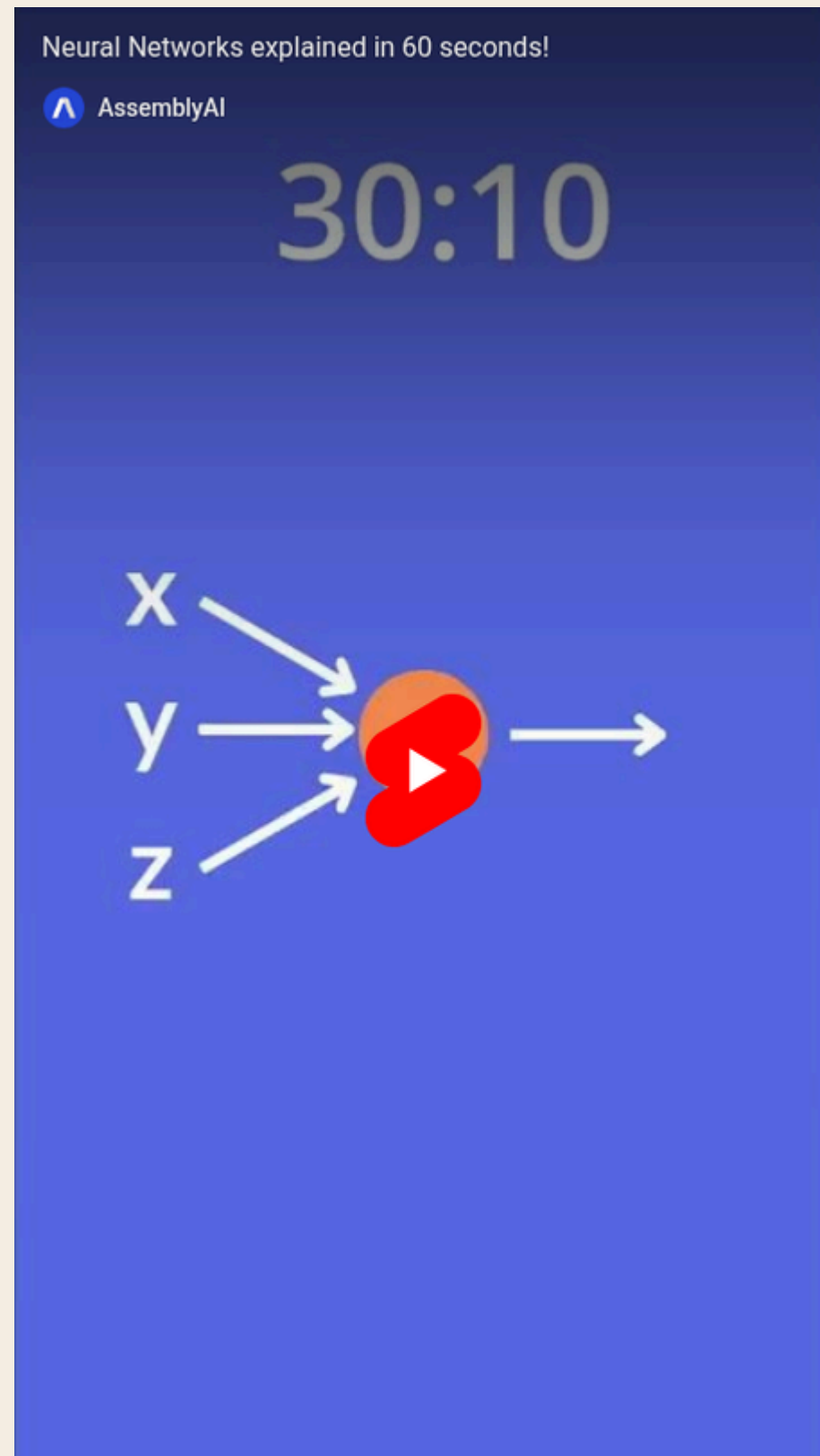
Feedforward:

- Pass inputs through the network layer by layer, left to right
- Each layer computes $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, then applies activation function $f(\mathbf{z})$
- Produces a prediction $\hat{y}$ at the output layer
- **Happens during both training and inference**

Backpropagation:

- Compute error between prediction $\hat{y}$ to true label y
- Send error signal backwards through the network, right to left
- Use **chain rule** to compute how much each weight contributed to the error $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$
- Update each weight using gradient descent to reduce error next time $w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$

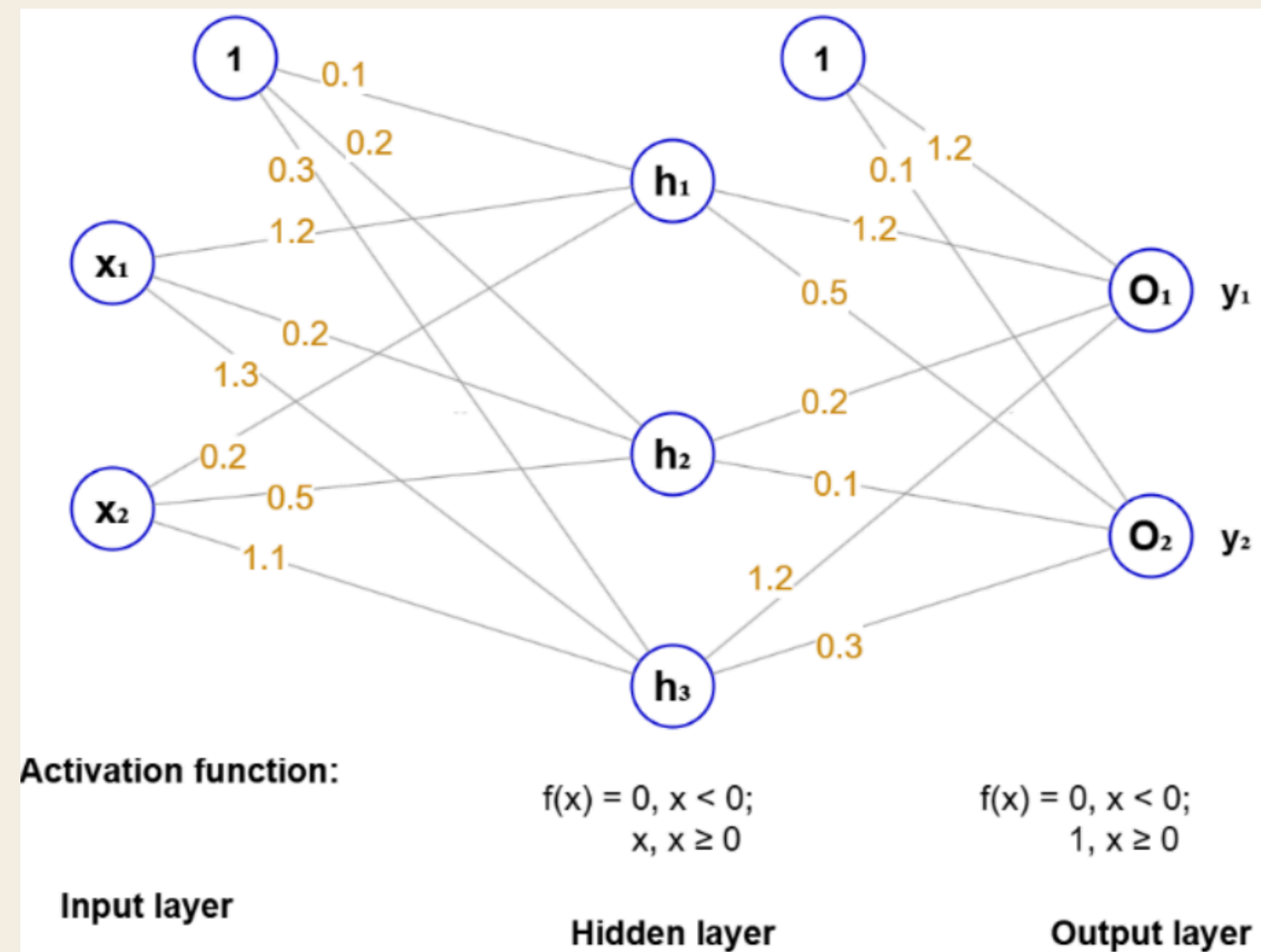
QUICK SUMMARY



TUTORIAL QUESTIONS

TUTORIAL QUESTION 1

Given the input vector $[x_1, x_2] = [1.2, 1.1]$ compute the output values: $[y_1, y_2]$ by performing a forward pass through the network.



Hidden layer:

$$h_1 = f(0.1 * 1 + 1.2 * 1.2 + 0.2 * 1.1) = 1.76$$

$$h_2 = f(0.2 * 1 + 0.2 * 1.2 + 0.5 * 1.1) = 0.99$$

$$h_3 = f(0.3 * 1 + 1.3 * 1.2 + 1.1 * 1.1) = 3.07$$

Output layer:

$$O_1 = 1.2 * 1 + 1.2 * 1.76 + 0.2 * 0.99 + 1.2 * 3.07 = 7.194$$

$$f(O_1) = 1$$

$$O_2 = 0.1 * 1 + 0.5 * 1.76 + 0.1 * 0.99 + 0.3 * 3.07 = 2.0$$

$$f(O_2) = 1$$

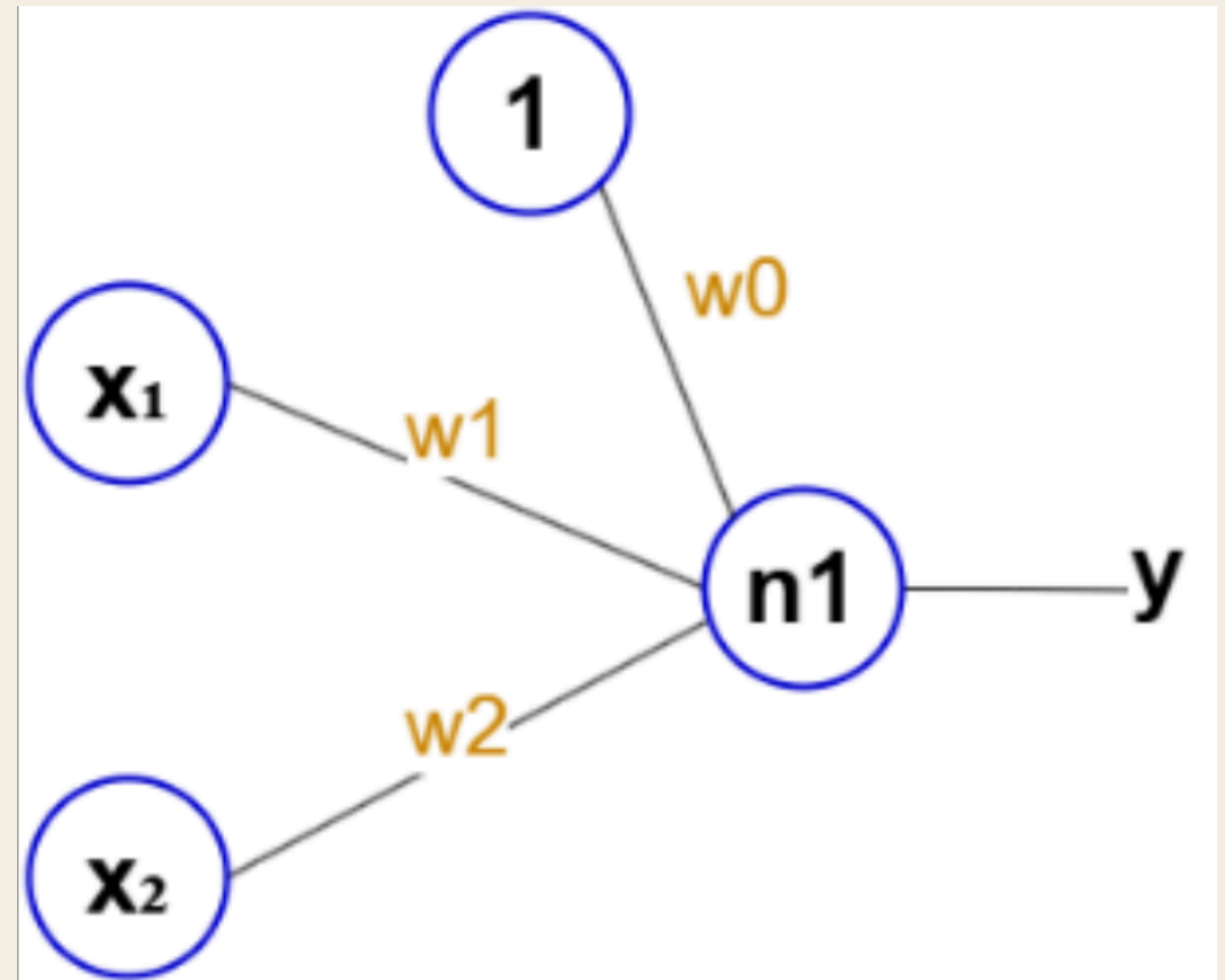
$$\text{Ans: } [y_1, y_2] = [1, 1]$$

TUTORIAL QUESTION 2A

Draw the neuron and label the inputs and weights.

The neuron has:

- (1) Inputs: x_1 , x_2
- (2) Bias input: with weight w_0
- (3) Output: y



TUTORIAL QUESTION 2B

Find weights for the neuron that compute the AND function.

x_1	x_2	$y = x_1 \text{ AND } x_2$
0	0	0
0	1	0
1	0	0
1	1	1

Activation Function:

$$f(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

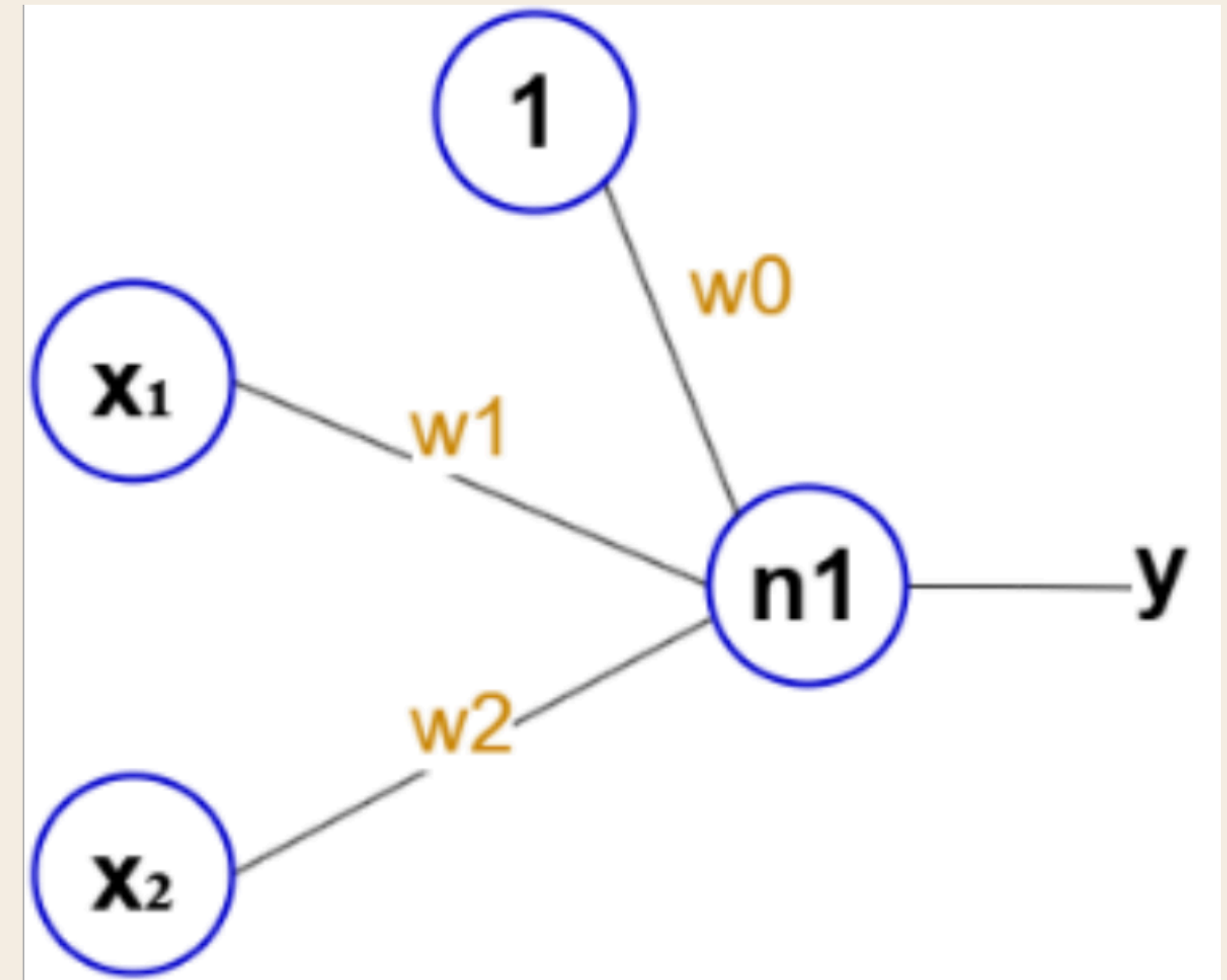
$$z = w_0 + w_1x_1 + w_2x_2$$

Some possible solutions:

$$[w_0, w_1, w_2] = [-2, 1, 1]$$

$$[w_0, w_1, w_2] = [-3, 2, 2]$$

- $w_0 < 0$
- $w_0 + w_1 < 0$
- $w_0 + w_2 < 0$
- $w_0 + w_1 + w_2 \geq 0$



TUTORIAL QUESTION 2C

Find weights for the neuron that compute the OR function.

x_1	x_2	$y = x_1 \text{ AND } x_2$
0	0	0
0	1	1
1	0	1
1	1	1

Activation Function:

$$f(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

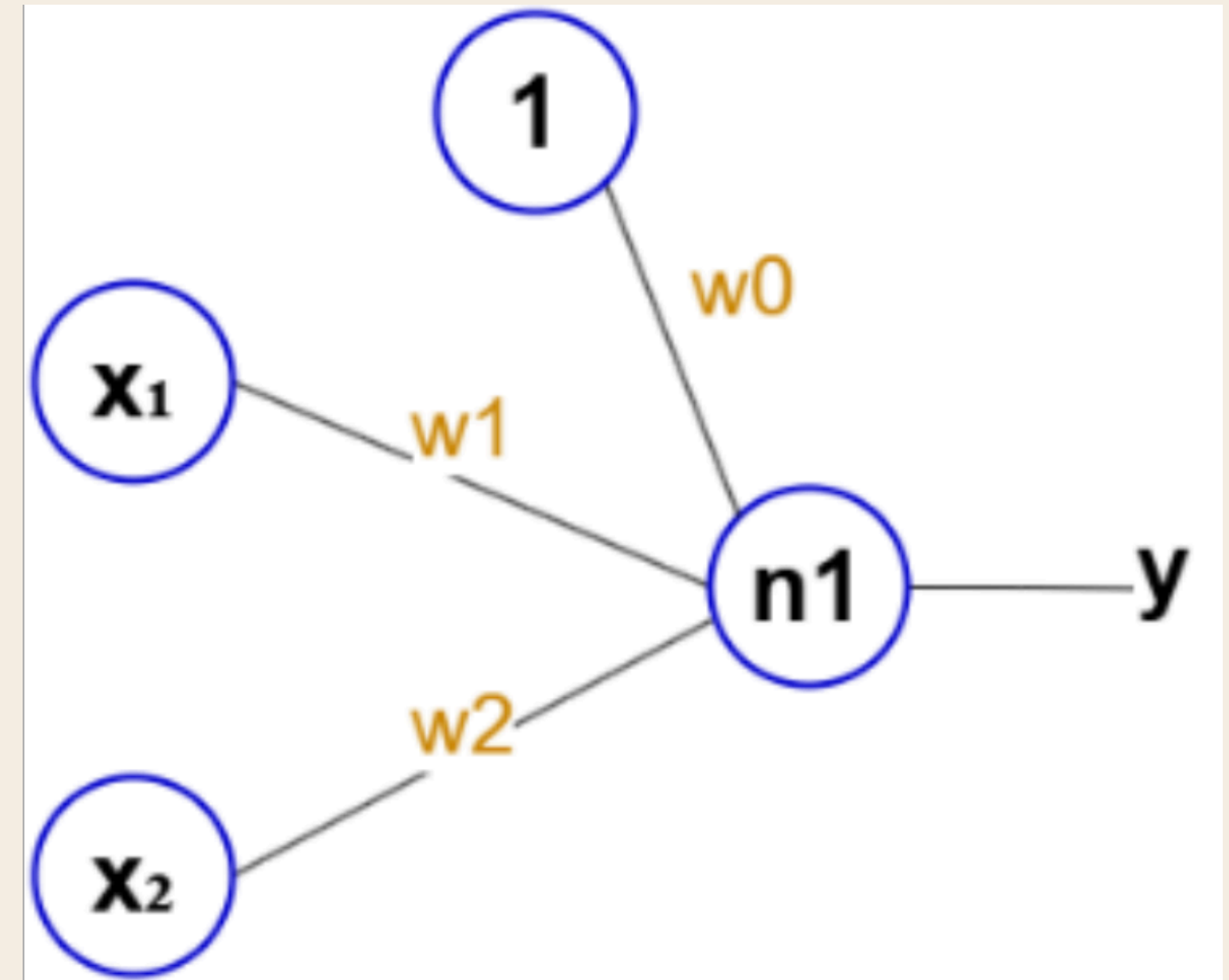
$$z = w_0 + w_1x_1 + w_2x_2$$

Some possible solutions:

$$[w_0, w_1, w_2] = [-1, 1, 1]$$

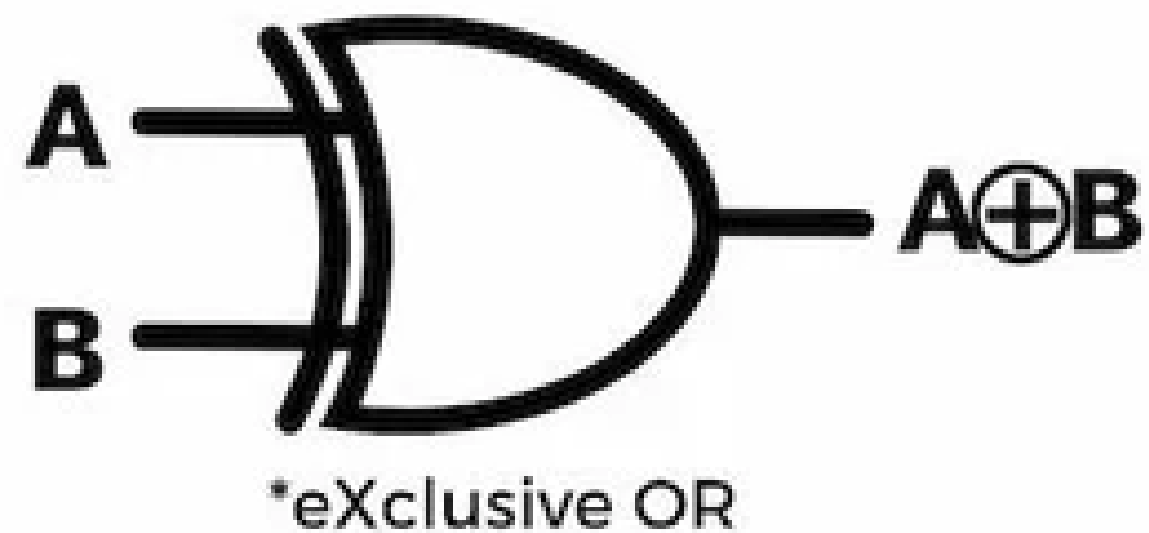
$$[w_0, w_1, w_2] = [-2, 2, 2]$$

- $w_0 < 0$
- $w_0 + w_2 \geq 0$
- $w_0 + w_1 \geq 0$
- $w_0 + w_1 + w_2 \geq 0$



TUTORIAL QUESTION 2D

Can a single neuron implement XOR?



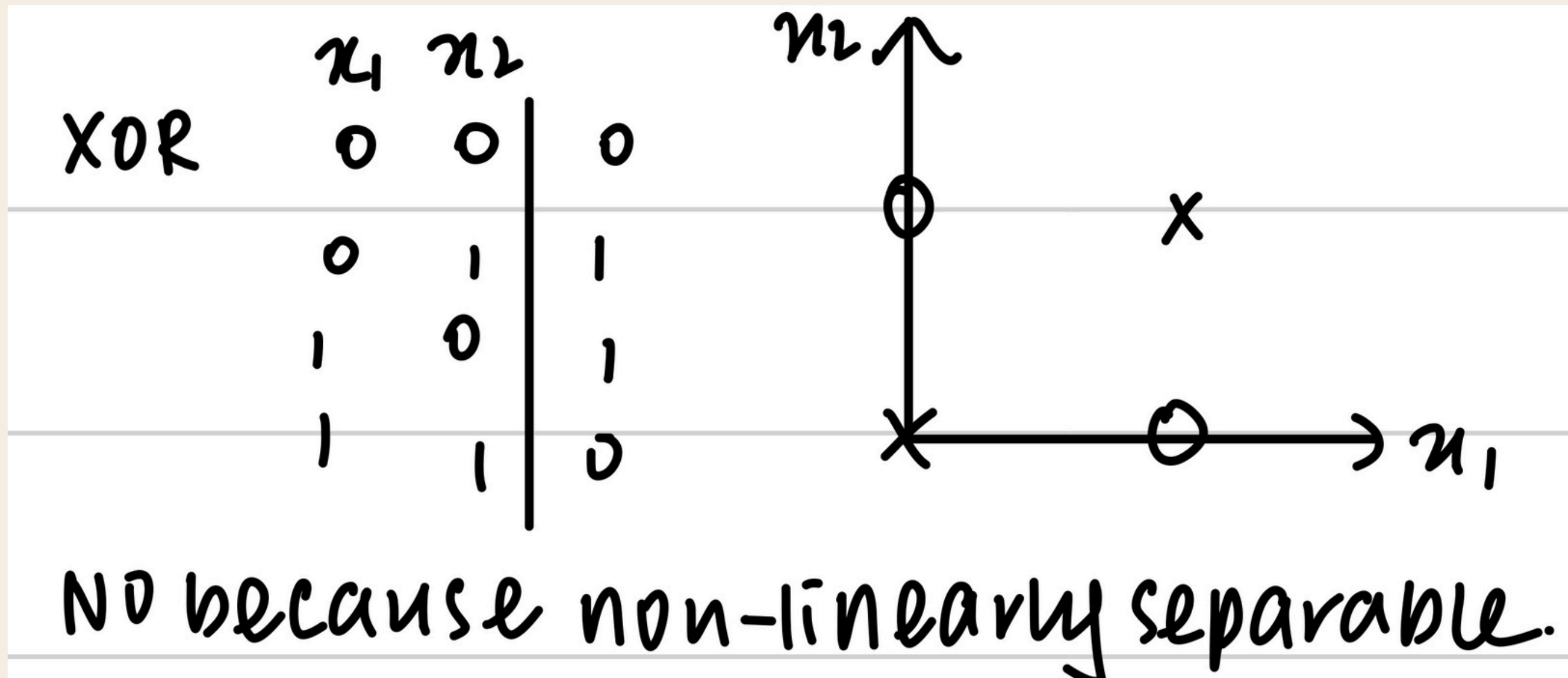
2 input XOR gate

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

What do you guys think?

TUTORIAL QUESTION 2D

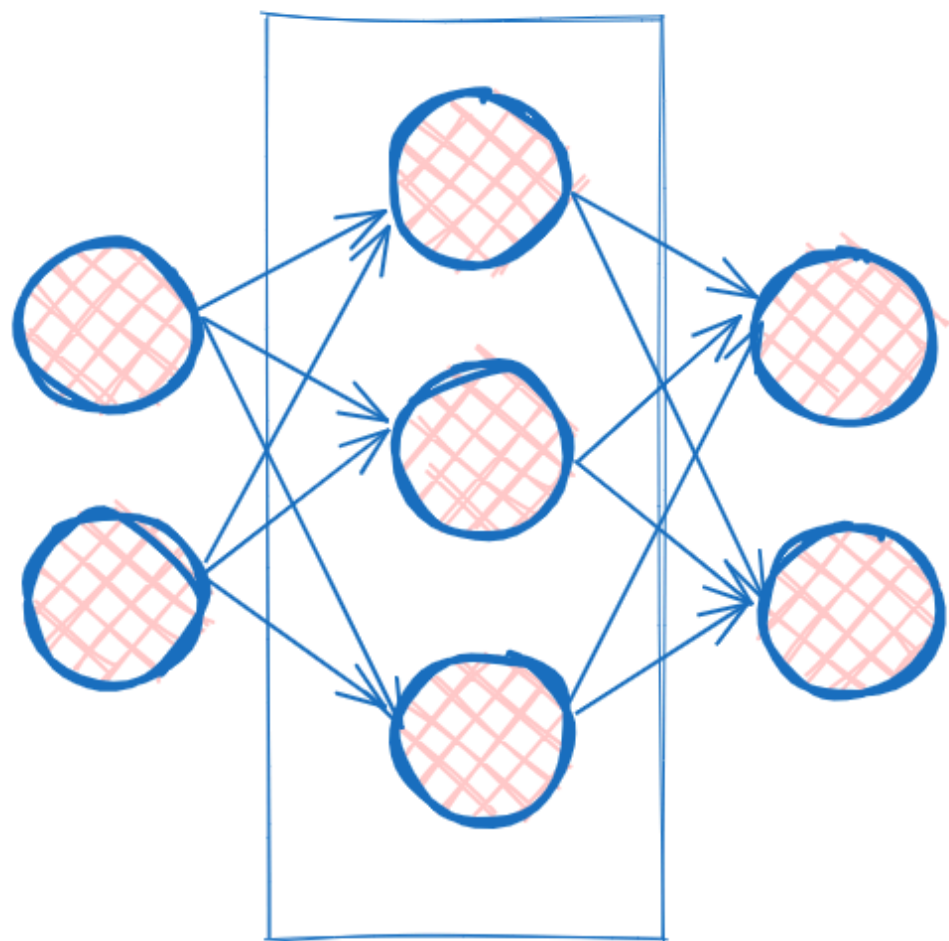
Can a single neuron implement XOR?



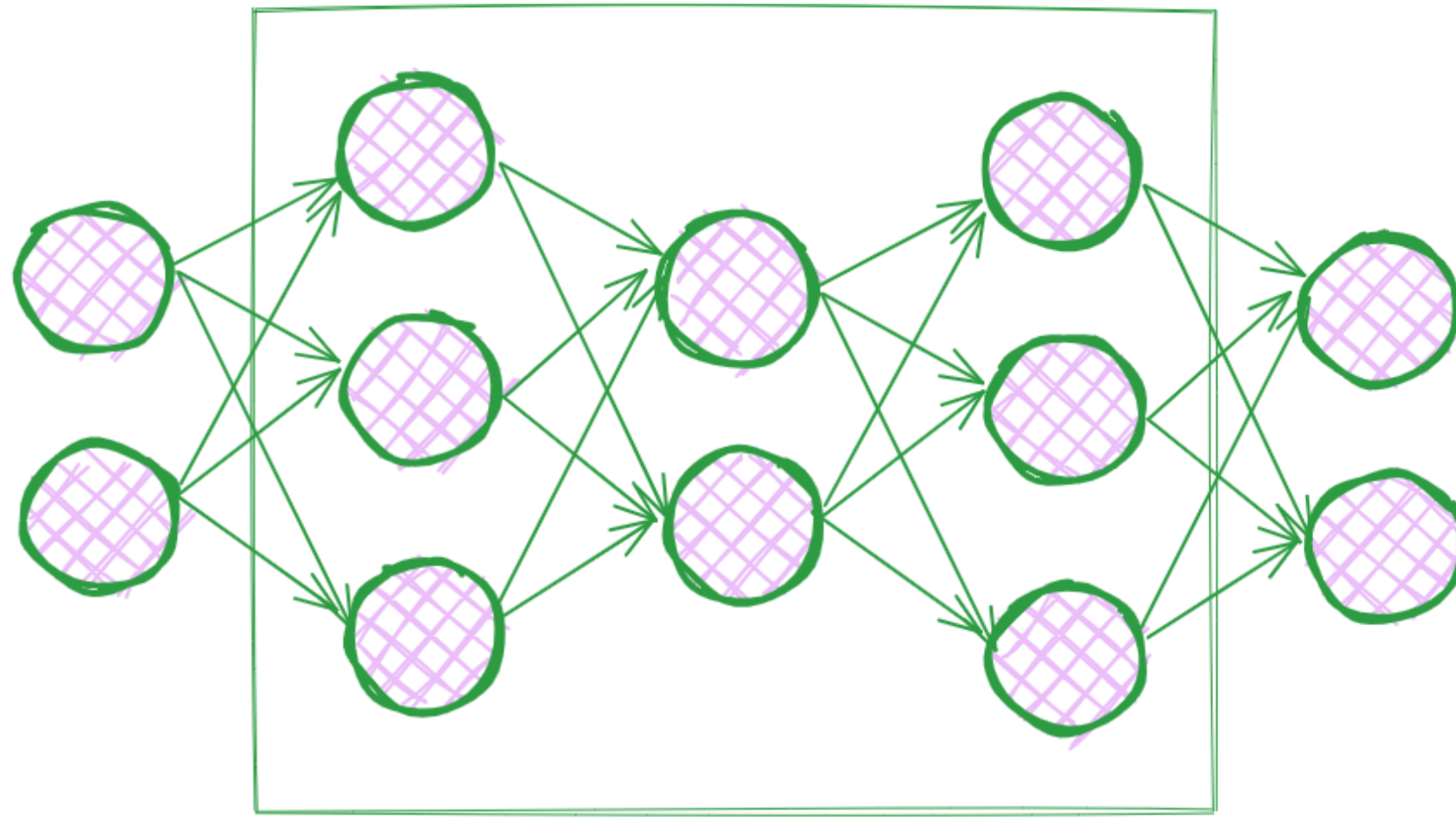
TUTORIAL QUESTION 3

What happens when we increase the number of hidden layers in a neural network?

Shallow neural network
with 1 hidden layer



Deep neural network
with multiple hidden layers

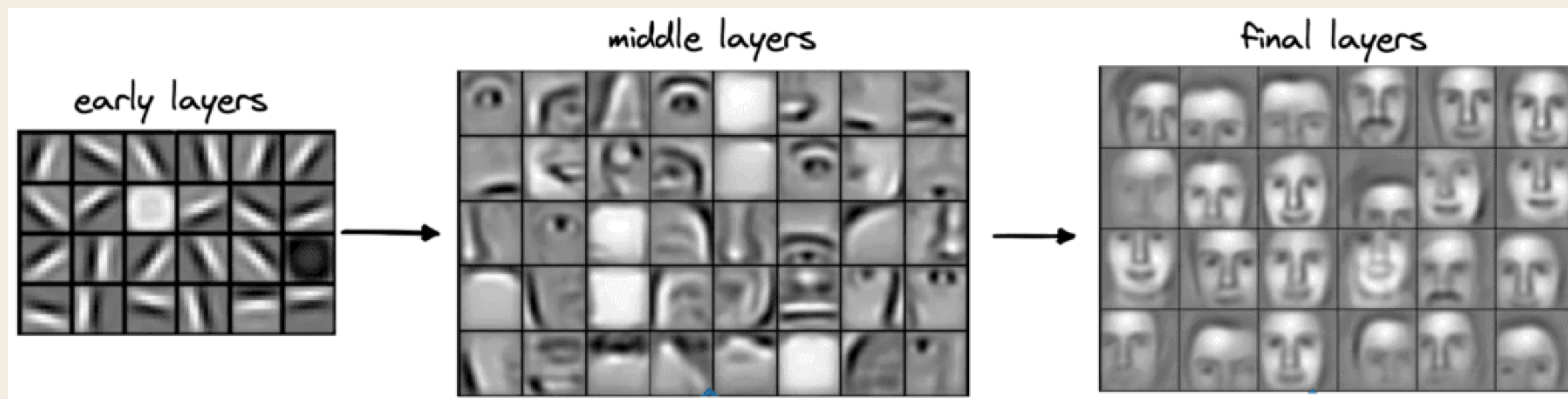


TUTORIAL QUESTION 3

What happens when we increase the number of hidden layers in a neural network?

Neural Network able to learn more complex patterns and representation

Eg. In Image Recognition Network: More layers allow the network to learn hierarchical representations



TUTORIAL QUESTION 3

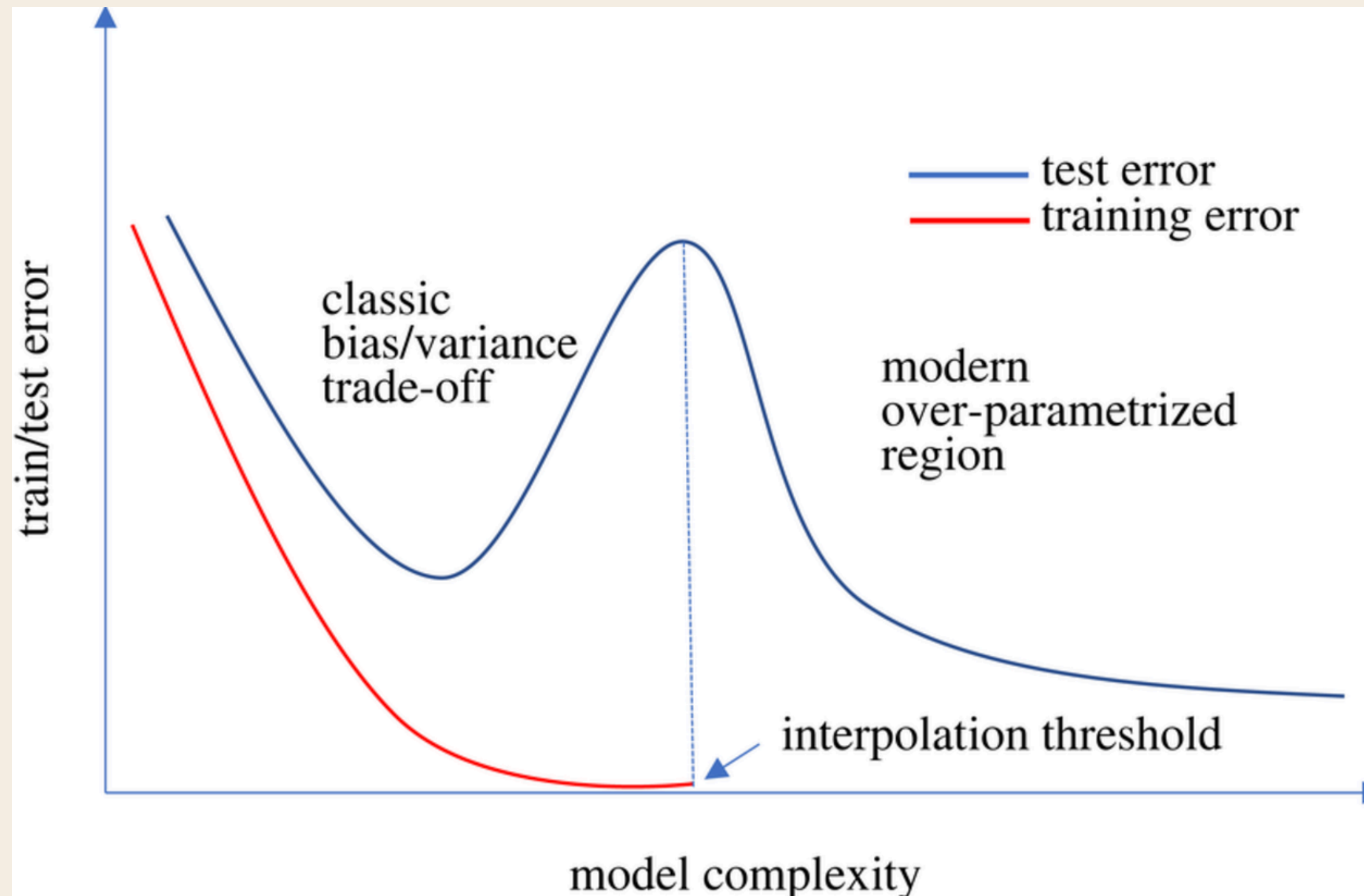
What happens when we increase the number of hidden layers in a neural network?

BUT increasing too much can lead to:

- Higher computational cost and longer training time
- Higher risk of overfitting
- Vanishing/exploding gradient problem: gradients shrink or grow exponentially across layers

TUTORIAL QUESTION 3 (EXTRA)

Double Descent Phenomenon



Before I said:

If we increase model complexity, we risk **overfitting** (because model “memorizes data”).

BUT if you keep increasing complexity past that threshold, test error rises sharply... then comes back down again?!

Eg. LLMs, Vision Transformers

TUTORIAL QUESTION 4

Explain the vanishing gradient and exploding gradient problems in neural networks.

Vanishing Gradient Problem

$$0.25^9 = 0.00000$$

$$w_{new} = w_{old} - \alpha \underbrace{\frac{\partial L}{\partial w_{old}}}_{\text{Close to } 0}$$

Weights hardly update at all

Exploding Gradient Problem

$$1.25^{15} = 28.42171$$

$$w_{new} = w_{old} - \alpha \underbrace{\frac{\partial L}{\partial w_{old}}}_{\text{A Big Value}}$$

Big and unstable weight updates

TUTORIAL QUESTION 4

Vanishing Gradient Problem:

- In backpropagation, the gradients often get **smaller** and approaches zero
- Weights in the lower hidden layers nearly unchanged and GD cannot converge to the optimum
- Usually associated with the choice of activation function (e.g. sigmoid)

One way to avoid this: replacing sigmoid/tanh activation functions with ReLU



TUTORIAL QUESTION 4

Exploding Gradient Problem:

- In contrast, the gradients can occasionally becomes **bigger and bigger**
- Large weight updates in the lower hidden layers and GD diverge
- Usually due to large initialised weights that leads to high derivatives, which causes significant deviations in new weight values from the previous one

One way to avoid this: Gradient Clipping

Formula

if $\|g\| > \tau$, clip:

$$g = g \times (\tau / \|g\|)$$

where

$$\|g\| = \sqrt{g_1^2 + g_2^2 + \dots + g_n^2}$$

τ = clipping threshold

if $\|g\| \leq \tau$, do nothing

Example ($\tau = 1.0$)

Gradients

$$g = [3, 4, 0]$$

Compute norm

$$\|g\| = \sqrt{9+16+0} = 5$$

Scale down

$$g \times (1.0 / 5)$$

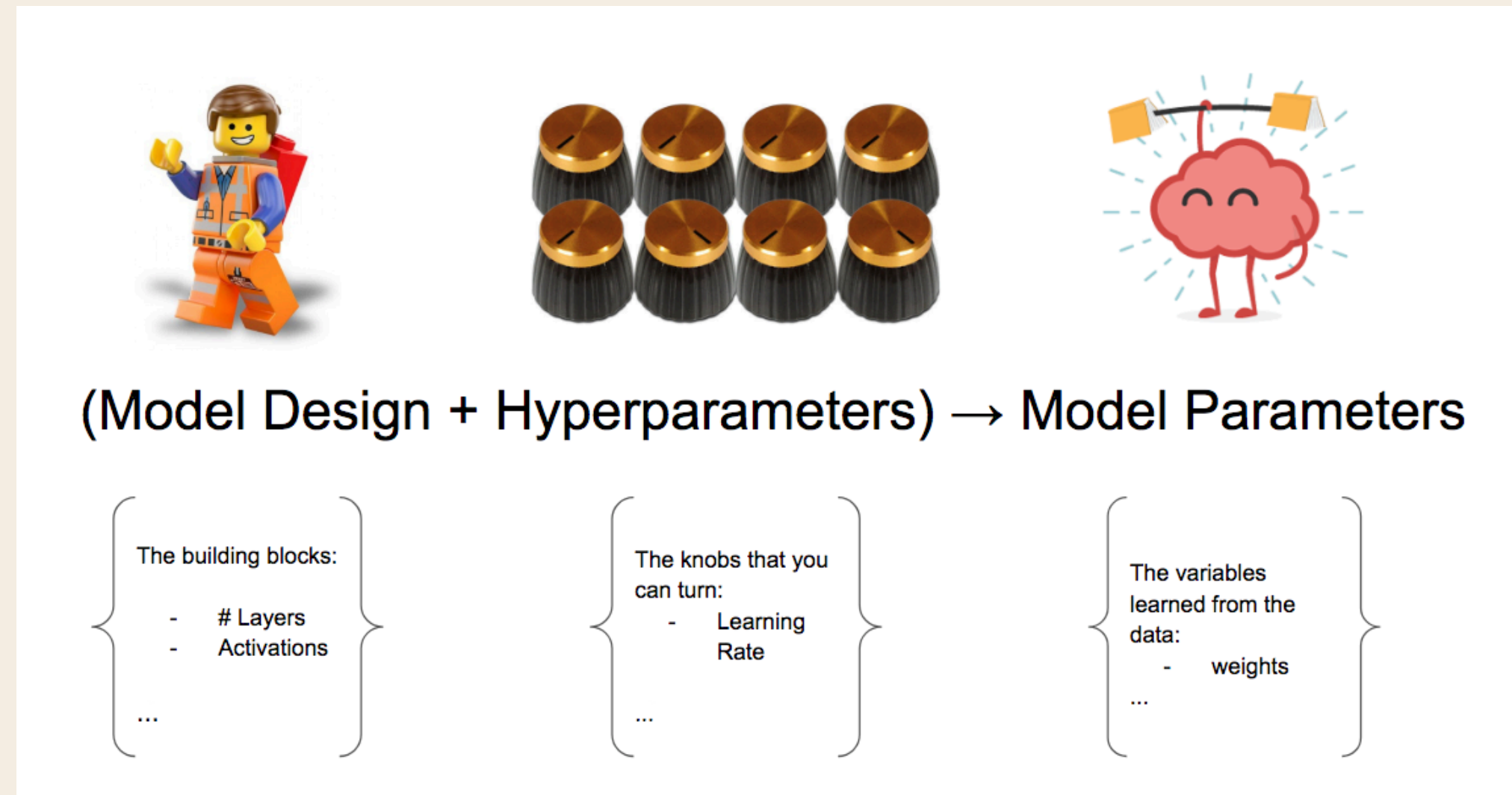
Result: $g_{\text{clipped}} = [0.6, 0.8, 0]$

Direction preserved — magnitude capped at $\tau = 1.0$

TUTORIAL QUESTION 5

What are hyperparameters in machine learning? List some common hyperparameters used in neural networks.

- **Model parameters** (weights) are learned from data.
- **Hyperparameters** are the "settings" that control how training behaves
 - defined by you before training!



TUTORIAL QUESTION 5

List some common hyperparameters used in neural networks.

- Train-test split ratio
- Learning rate in optimization algorithms (e.g., Gradient Descent)
- Choice of optimization algorithm (SGD, etc.)
- Activation function used in neural network layers (ReLU, Sigmoid, Tanh)
- Number of hidden layers
- Number of neurons in each layer
- Number of training epochs
- Number of clusters (k) in K-means clustering
- Number of neighbours (k) in KNN

POLLEV QUESTIONS

Join by Web:

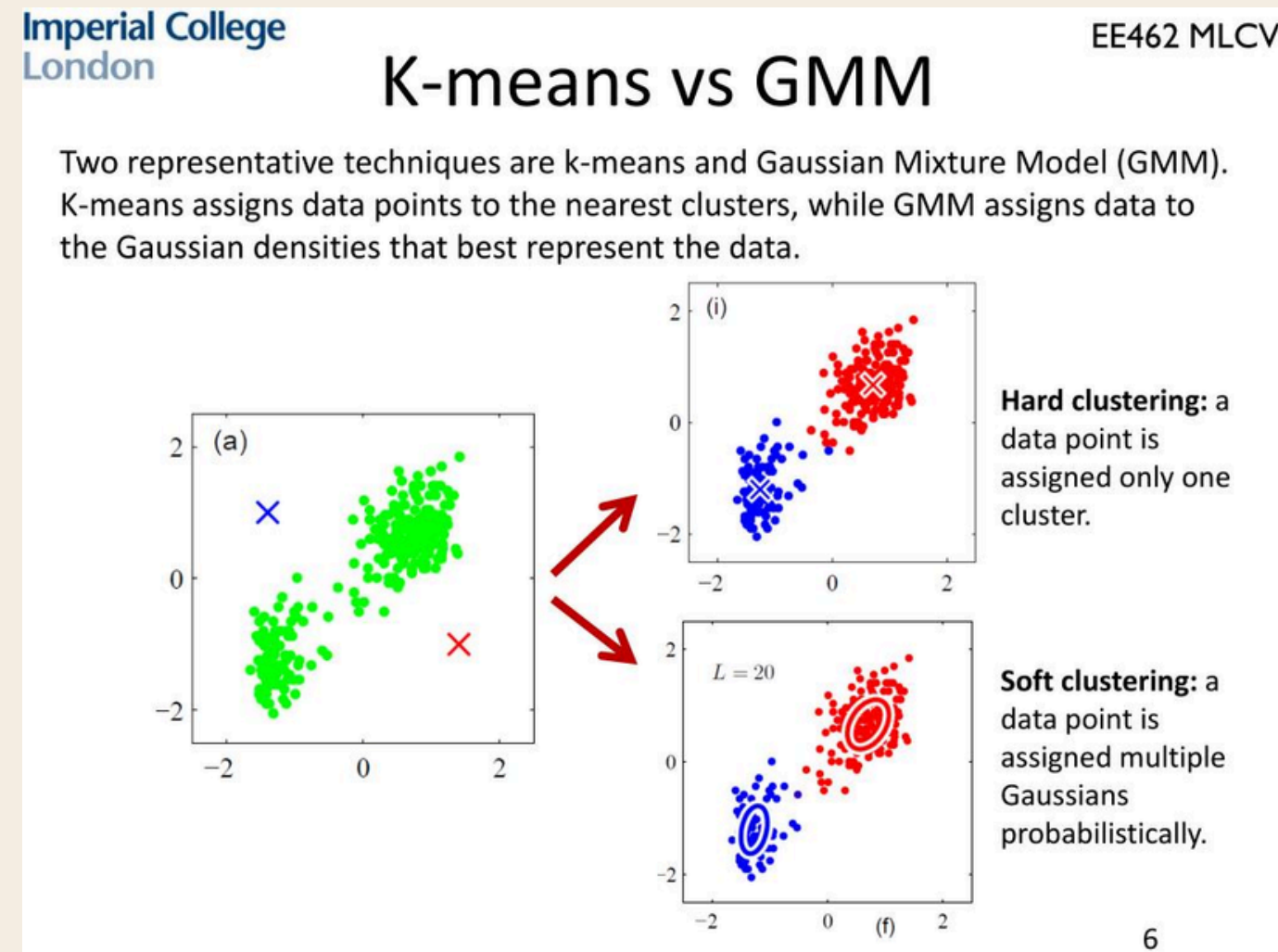
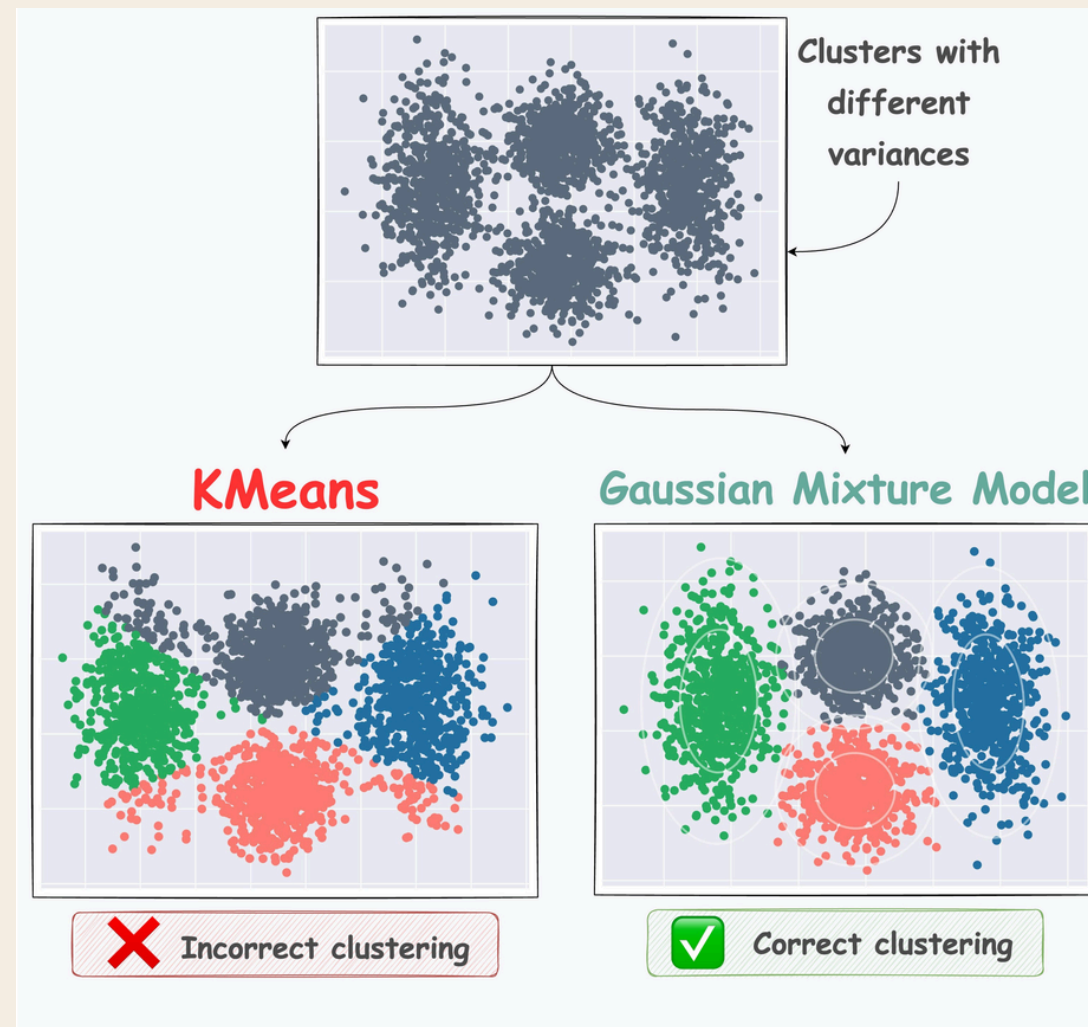
PollEv.com/kaironglee

Join by QR:



EXTRA READINGS (LAST WEEK)

- Another type of Clustering: Gaussian Mixture Models
- Intro to Dimensionality Reduction
- Deeper into Dimensionality Reduction: t-SNE, UMAP



EXTRA READINGS FOR NN

- (Video) Detailed explanation of backpropagation by 3Blue1Brown [here](#).
- (Video) ReLU is important - here's [why](#).
- [ReLU, Leaky ReLU, and PReLU](#)
- [\(Video\) Vanishing and Exploding Gradient --> Touch on a bit RNN](#)

NEXT AND LAST WEEK :(

CNN and RNN

